

DataPrepToolkit

Official User Guide & Technical Reference Manual

Version 1.1.0 | Python ≥ 3.11 | Pure Local & Offline Execution | 100% Deterministic

Executive Overview: DataPrepToolkit is a production-grade Python library designed for automated dataset profiling, data quality assessment, safe cleaning, rule validation, outlier detection, memory optimization, and quality reporting. It acts as the trustworthy foundation layer for downstream Exploratory Data Analysis (AutoEDA), analytics (AutoAnalytics), and dashboards (AutoBI).

1. Core Philosophy & Ecosystem Architecture

DataPrepToolkit strictly operates as an independent, deterministic data preparation layer. Input DataFrames are **never mutated in-place**; every function returns a new DataFrame alongside a structured audit result object (e.g., CleaningResult, OptimizationResult, ValidationResult).

Layer	Ecosystem Module	Primary Responsibility
Layer 1	DataPrepToolkit	Data Hygiene, Quality Assessment, Cleaning, Validation, Optimization
Layer 2	AutoEDA	Exploratory Analysis, Statistical Distributions, Visual Correlations
Layer 3	AutoAnalytics	Advanced Diagnostics, Hypothesis Testing, Analytical Conclusions
Layer 4	AutoBI	Interactive Decision Dashboards & Business Communication

2. Installation & Quick Start

Install DataPrepToolkit via standard pip:

```
pip install datapreptoolkit
```

30-Second Executable Workflow:

```
import pandas as pd
import datapreptoolkit as dpt

# 1. Load CSV data safely
df = dpt.load_csv('sales.csv')

# 2. Profile dataset & generate quality score
profile = dpt.profile_dataset(df)
report = dpt.generate_quality_report(df)
print(f'Quality Score: {report.overall_quality_score}/100')

# 3. Run full cleaning pipeline (imputation + deduplication + datetimes)
cleaned_df, clean_res = dpt.clean_dataset(df)

# 4. Validate business constraints
rules = [dpt.ValidationRule(column='price', rule_type='range', min_value=0.01)]
val_res = dpt.validate_dataset(cleaned_df, rules)

# 5. Export professional HTML report
dpt.export_html_report(report, 'quality_report.html')
```

3. Real-World E-Commerce Scenario Walkthrough

Below is a demonstration of processing an unrefined retail order table containing duplicate transactions, missing prices, invalid negative quantities, and unoptimized memory types.

Metric / Indicator	Raw Initial State	Prepared Final State	Action Taken / Impact
Row Count	8 rows	6 rows	Removed 1 duplicate row & 1 invalid negative row
Column Count	6 columns	6 columns	All required schema columns preserved
Missing Cells	3 missing cells	0 missing cells	Median imputed for price, mode for region
Duplicate Rows	1 duplicate row	0 duplicate rows	Deduplicated on primary order key
Datetime Types	0 (object string)	1 (datetime64[ns])	Parsed order_date to timestamp cleanly
Memory Footprint	900 Bytes	420 Bytes	Down-casted int/float and string to category (53.3% savings)
Quality Score	78.20 / 100	98.50 / 100	+20.30 Quality Score Improvement

4. Feature Guides & Strategy Reference

A. Missing Value Imputation Strategies

handle_missing_values() supports 10 distinct imputation strategies. When using strategy='drop_column', columns are dropped **only** if their null ratio exceeds drop_threshold (default 0.6 = 60%). Columns below the threshold safely fall back to mode imputation.

Strategy	Target Type	Behavior Description
----------	-------------	----------------------

median (default)	Numeric	Fills numeric nulls with median. Non-numeric columns fall back to mode.
mean	Symmetric Numeric	Fills numeric nulls with mean. Non-numeric columns fall back to mode.
mode	Categorical / String	Fills nulls with the single most frequent value.
ffill / bfill	Time-Series	Forward-fills or backward-fills missing values sequentially.
interpolate	Ordered Numeric	Linear interpolation for numeric; forward-fill for categoricals.
drop_rows	Critical Target	Drops any row containing a null value in specified columns.
drop_column	Heavy Missing (>60%)	Drops column if null ratio > threshold. Otherwise imputes with mode.
zero / empty	Sparse / Text	Fills numeric nulls with 0 and categorical nulls with 'Unknown'.

B. Outlier Detection Mechanics

DataPrepToolkit provides read-only outlier detection via IQR and Z-Score (Standard and Modified MAD-based). Infinities (inf / -inf) are isolated so they do not corrupt finite bounds.

IMPORTANT RULE: Outlier Detection ≠ Outlier Deletion.

DataPrepToolkit flags outlier row indices programmatically without deleting rows. An outlier may represent a high-value transaction or fraud signal that must be analyzed rather than deleted.

C. Rule-Based Validation System

Declarative validation via ValidationRule checks DataFrames against business constraints and returns a ValidationResult detailing any ValidationViolations.

```
from datapreptoolkit import validate_dataset, ValidationRule

rules = [
    ValidationRule(column='order_id', rule_type='required'),
    ValidationRule(column='order_id', rule_type='no_duplicates'),
    ValidationRule(column='age', rule_type='range', min_value=18, max_value=65),
    ValidationRule(column='email', rule_type='regex', pattern=r'^[\w.-]+@[\w.-]+\.\w+$'),
    ValidationRule(column='status', rule_type='in_set', allowed_values={'active', 'pending'}),
]
result = validate_dataset(df, rules)
```

5. Configuration & Quality Scoring Mechanics

The composite Quality Score (0–100) starts at 100.0 and deducts weighted penalties defined in ToolkitConfig.quality_weights:

Quality Score Formula:

$$\text{Score} = 100.0 - [(\text{missing} / \text{total_cells}) \times W_{\text{missing}}] - [(\text{duplicates} / \text{rows}) \times W_{\text{dup}}] - [N_{\text{const}} \times W_{\text{const}}] - [N_{\text{high_card}} \times W_{\text{card}}] - [N_{\text{outlier}} \times W_{\text{outlier}}]$$

```
from datapreptoolkit import ToolkitConfig, EncodingStrategy, ZScoreMethod

config = ToolkitConfig(
    remove_duplicates=True,
    parse_datetimes=True,
    optimise_memory=True,
    detect_outliers=True,
    outlier_method='zscore',
    zscore_method=ZScoreMethod.MODIFIED,
    encoding_strategy=EncodingStrategy.LABEL,
    high_cardinality_threshold=0.95,
    constant_threshold=0.99,
    quality_weights={
        'missing': 40.0, 'duplicate': 20.0, 'constant': 2.0,
        'high_cardinality': 1.0, 'outlier': 3.0,
    },
)
```

6. Feature Decision Matrix & API Catalog

Goal / Task	Function to Call	Returned Object Type
Load CSV safely	load_csv(filepath)	pd.DataFrame
Profile shape & metadata	profile_dataset(df)	DatasetProfile
Analyze missing cells	analyze_missing_values(df)	MissingValueAnalysis
Impute missing cells	handle_missing_values(df)	(pd.DataFrame, CleaningResult)
Deduplicate rows	remove_duplicates(df)	(pd.DataFrame, CleaningResult)
Detect invalid values	detect_invalid_values(df)	(dict, CleaningResult)
Detect outliers	detect_outliers(df)	OutlierDetection
Downcast memory	optimise_memory(df)	(pd.DataFrame, OptimizationResult)
Validate data rules	validate_dataset(df, rules)	ValidationResult
Generate quality score	generate_quality_report(df)	QualityReport
Export HTML report	export_html_report(report)	str (File Path)

7. Exception Hierarchy & Hardened Robustness

```
DataPrepError (Base)
■■■ LoadError (FileFormatError, EmptyDatasetError)
■■■ CleaningError (InvalidColumnError, IncompatibleDataError)
■■■ ValidationError (QualityThresholdError)
■■■ ReportError
■■■ ConfigError
```

Verified Edge Case Handling: DataPrepToolkit has been systematically tested across 20 distinct dataset archetypes (Datasets A–T). It cleanly handles empty DataFrames (0 rows), single-row DataFrames, single-column DataFrames, all-null columns, constant columns, extreme outliers, infinite values (inf/-inf), mixed `np.nan`/`None`

representations, and non-English UTF-8 headers (Arabic, Spanish, Japanese) without unhandled crashes.